

Programación I

Vectores

*Estructura de datos para almacenar
múltiples valores*

Clasificación de las variables según su dimensión

Así como clasificamos las variables por su tipo de dato, también podemos hacerlo por su **dimensión**:

- **Variables simples (escalares):** Almacenan un solo valor a la vez.
Ejemplo: `int edad = 25;`
- **Objetos:** Instancias de clases. Pueden contener múltiples atributos y comportamientos.
- **Arrays (vectores y matrices):** Variables que aceptan varios valores a la vez.
 - **Vector:** array unidimensional.
 - **Matriz:** array multidimensional.

¿Qué es un vector?

Un **vector** es una estructura de datos que permite almacenar **múltiples valores** dentro de una **misma variable**, todos del mismo tipo.

Reglas importantes:

1. **Tamaño conocido:** Debe definirse antes de usar el vector.
2. **Indexación base 0:** El primer elemento está en la posición 0.
3. **Elementos homogéneos:** Todos los elementos deben ser del mismo tipo de dato (enteros, flotantes, booleanos, etc.).

Declaración de un vector

Para declarar un vector en C/C++ se usa la siguiente sintaxis:

```
tipo nombre[tamaño];
```

Ejemplo:

```
int mi_vector[10];
```

- **int** → tipo de dato de los elementos
- **mi_vector** → nombre del vector
- **[10]** → tamaño del vector (10 elementos)

El tamaño debe ser un **número natural mayor a 0 y constante** (no puede ser una variable común).

Declaración de un vector

Podemos usar una constante para definir el tamaño:

```
const int TAM = 10;  
float mi_vector[TAM];
```

Esto crea un vector de 10 elementos de tipo float.

Al compilar, el vector reserva espacio para 10 valores.

Acceso a elementos del vector

Para acceder a un elemento del vector, usamos su **índice** entre corchetes.

```
int pos = 3;
int vec[5];

cout << vec[2];    // muestra el tercer elemento
cin >> vec[4];     // asigna un valor al quinto elemento
vec[pos] = 100;    // asigna 100 al cuarto elemento
```

Recuerda:

El primer elemento está en `vec[0]`

El último está en `vec[tamaño - 1]`

Índice	int vec[8]
0	
1	
2	
3	
4	

Representación del
vector `vec[5]`

Ejemplo de acceso y operaciones

Para acceder a un elemento del vector, usamos su **índice** entre corchetes.

```
int valor_0, valor_1, valor_2;
int vec[4] = {10, 20, 30, 40};
Valor_0 = vec[1];           // 20
valor_1 = vec[0] * vec[3];  // 10 * 40 = 400
if (vec[0] > 0) {
    valor_2 = vec[0];       // 10
}
vec[1]++;                   // ahora vec[1] es 21
```

Índice	int vec[4]
0	
1	
2	
3	

Variable	Contiene
valor_0	
valor_1	
valor_2	

Inicialización de vectores

Si no inicializamos un vector, sus elementos contendrán basura (valores aleatorios).

Formas de inicializar todo a cero:

```
// Usando un bucle
int i;
int vec[10];
for (i = 0; i < 10; i++) {
    vec[i] = 0;
}
```

```
// Forma compacta (C/C++)
int vec[10] = {};
```


Ejercicios propuestos

1. Declara un vector de 5 enteros e inicialízalo con los números del 1 al 5.
2. Muestra en pantalla el primer y el último elemento del vector anterior.
3. Declara un vector de 10 flotantes, inicialízalos todos en 0.0 y luego asigna el valor 7.5 al cuarto elemento.
4. Crea un vector de 8 enteros, pide al usuario que ingrese valores para cada posición y luego muestra la suma total.
5. Declara una constante TAM = 6, crea un vector de ese tamaño y llena sus posiciones con los primeros 6 números pares mayores a cero.